

Tab 1

AGUADAGNO

Vuoi lavorare con me? 🤝 <https://antonioguadagno.it/>

Unisciti al gruppo **Telegram** (gratis) 🤝 https://t.me/antonio_guadagno_ai

Iscriviti alla **newsletter** (gratis) 🤝 <https://antonioguadagno.substack.com>

Seguimi su **Instagram** 🤝 https://www.instagram.com/antonio_guadagno_ita

Seguimi su **LinkedIn** 🤝 <https://www.linkedin.com/in/antonio-guadagno-ab9ba72b1>

In questa mini-guida ti spiego quali sono i principali comandi disponibili in Claude Code e cosa fa ciascuno di essi. Leggendola, ti farai una chiara idea del potenziale di questo strumento. Potresti anche scoprire che il progetto che hai in mente, e che ritenevi impossibile, si può realizzare più facilmente di quanto pensi.

1. Setup e configurazione

- **NOME COMANDO:** `/model`
A COSA SERVE: Selezionare il modello che vuoi utilizzare (Opus, Sonnet, Haiku). Cliccando sulla freccia a destra e sulla freccia a sinistra, puoi regolare il livello di effort (basso, medio, alto, max).
UN ESEMPIO CONCRETO: Lanci `/model`, scegli Opus e alzi l'effort su "alto" per un refactor complesso dell'architettura.
- **NOME COMANDO:** `/effort`
A COSA SERVE: Cambiare al volo il livello di reasoning (low, medium, high, xhigh, max) senza toccare il modello, tramite uno slider a frecce. Utile per dosare i token.
UN ESEMPIO CONCRETO: Fai domande semplici con `/effort medium`, poi alzi a `xhigh` quando arrivi a un problema complesso.
- **NOME COMANDO:** `/init`
A COSA SERVE: Analizzare il codice del progetto e generare un file `CLAUDE.md` che descrive come funziona l'app. Questo file viene riletto a ogni sessione in quella cartella. Ci sono pro e contro: la rilettura aiuta Claude ad essere più consapevole ma brucia token.
UN ESEMPIO CONCRETO: Apri Claude Code in una nuova repo, lanci `/init` e ottieni un `CLAUDE.md` con architettura, moduli e convenzioni seguite dagli sviluppatori.
- **NOME COMANDO:** `/memory`
A COSA SERVE: Modificare a mano i file `CLAUDE.md` e gestire l'auto-memory. È il "passo due" di `/init` per rifinire il contesto invece di subirlo.
UN ESEMPIO CONCRETO: Dopo `/init` lanci `/memory` e togli una sezione troppo verbosa che ti appesantiva il contesto a ogni avvio.
- **NOME COMANDO:** `/permissions`
A COSA SERVE: Gestire le regole `allow/ask/deny` dei permessi sui tool. Permette a Claude di lavorare in autonomia senza approvare ogni azione, ma in sicurezza. Fondamentale con agenti, `/goal` e `/loop`.
UN ESEMPIO CONCRETO: Lanci `/permissions` e autorizzi sempre `git status` e l'esecuzione dei test, così Claude smette di chiederti conferma a ogni passo.

- **NOME COMANDO:** /mcp
A COSA SERVE: Gestire le connessioni ai server MCP e l'autenticazione OAuth: colleghi, riconnetti, abiliti o disabiliti gli strumenti esterni che agenti e sessioni useranno.
UN ESEMPIO CONCRETO: Lanci /mcp, riconnetti il server GitHub scollegato e torni a far leggere a Claude le tue issue e PR. Similmente puoi connettere Claude Code a Gmail, Drive, Hubspot e centinaia di altre applicazioni.
- **NOME COMANDO:** /agents
A COSA SERVE: Creare e gestire agenti personalizzati per attività specifiche. Li configuri una volta (descrizione, tool, modello, memoria) e li riusi all'infinito.
UN ESEMPIO CONCRETO: Crei "Architect", un agente su Opus 4.8 che progetta l'architettura delle app, e lo richiami ogni volta che ti serve con "Run Task".

2. Gestione del contesto e della sessione

- **NOME COMANDO:** /context
A COSA SERVE: Vedere quale percentuale della finestra di contesto stai occupando, con una griglia che evidenzia cosa pesa di più (tool, memoria, prompt, skill). Aiuta a prevenire il context rot (e cioè il calo di qualità delle risposte quando il contesto è troppo pieno).
UN ESEMPIO CONCRETO: Dopo un'ora di utilizzo di Claude Code lanci /context, vedi che sei al 62% e capisci che è il momento di alleggerire.
- **NOME COMANDO:** /clear
A COSA SERVE: Cancellare tutto e ripartire con contesto vuoto. La chat precedente resta recuperabile tramite il comando /resume e le modifiche ai file restano. Da usare quando cambi task.
UN ESEMPIO CONCRETO: Finito il modulo login della tua app, devi passare alla dashboard (cosa scollegata): lanci /clear e riparti pulito.
- **NOME COMANDO:** /compact
A COSA SERVE: Liberare spazio riassumendo la conversazione senza far dimenticare a Claude ciò che è stato detto. La via di mezzo tra continuare e ricominciare.
UN ESEMPIO CONCRETO: Sei al 70% di contesto ma stai ancora sullo stesso task: lanci /compact per liberare token mantenendo il filo.

- **NOME COMANDO:** /resume
A COSA SERVE: Recuperare una conversazione precedente per ID o nome, o aprire un picker. È il complemento di /clear: la chat "cancellata" la ritrovi qui. Alias: /continue.
UN ESEMPIO CONCRETO: Ieri sistemavi i pagamenti sulla tua app, oggi lanci /resume, selezioni quella sessione e riprendi da dove avevi lasciato.
- **NOME COMANDO:** /rewind
A COSA SERVE: Tornare a un punto precedente della conversazione e/o del codice, come un rollback. Utile sia se cancelli una chat per sbaglio sia per annullare modifiche sbagliate al codice.
UN ESEMPIO CONCRETO: Claude ha modificato cinque file e ha rotto la build: lanci /rewind, scegli il checkpoint di prima e ripristini tutto.

3. Pianificazione e lavoro autonomo

- **NOME COMANDO:** /plan
A COSA SERVE: Chiedere a Claude di pianificare prima di toccare il codice. Propone le azioni e attende il tuo via, così sai quali file modificherà e in che ordine.
UN ESEMPIO CONCRETO: /plan fix the auth bug: Claude espone il piano, tu lo approvi e solo allora inizia a scrivere.
- **NOME COMANDO:** /goal
A COSA SERVE: Assegnare un obiettivo con una condizione verificabile; Claude lavora turno dopo turno finché non è raggiunta, controllato da un modello valutatore. È la base degli agentic loop.
UN ESEMPIO CONCRETO: /goal tutti i test della suite passano senza skip: Claude legge i test falliti, corregge, rilancia e ripete finché sono verdi.
- **NOME COMANDO:** /loop
A COSA SERVE: Eseguire un prompt a una certa cadenza finché la sessione resta aperta, per delegare controlli ricorrenti.
UN ESEMPIO CONCRETO: /loop 5m check if the deploy finished: ogni 5 minuti Claude verifica se il deploy è andato a buon fine.
-

- **NOME COMANDO:** `/batch`
A COSA SERVE: Scomporre una modifica grande in 5-30 unità indipendenti e lanciare un subagent in background per ciascuna, in git worktree isolati con test e PR. Il salto da un agente a tanti in parallelo.
UN ESEMPIO CONCRETO: `/batch migrate src/` from Solid to React: Claude propone il piano diviso in unità e le lavora in parallelo aprendo una PR per ognuna.
- **NOME COMANDO:** `/branch`
A COSA SERVE: Creare una copia della conversazione e spostarti dentro, per provare una strada diversa senza perdere l'originale (a cui torni con `/resume`).
UN ESEMPIO CONCRETO: Prima di un refactor rischioso lanci `/branch tentativo-redux`, esplori l'idea e, se non funziona, torni alla versione di partenza.
- **NOME COMANDO:** `/fork`
A COSA SERVE: Lanciare un subagent in background che eredita tutta la conversazione e lavora su un task mentre tu prosegui; il risultato rientra nella tua chat quando ha finito.
UN ESEMPIO CONCRETO: Mentre discuti l'architettura, lanci `/fork` scrivi i test per il modulo auth e l'agente te li prepara in parallelo.
- **NOME COMANDO:** `/background`
A COSA SERVE: Staccare l'intera sessione e farla girare come background agent, liberandoti il terminale. Per attività lunghe che non vuoi presidiare. Alias: `/bg`.
UN ESEMPIO CONCRETO: Hai avviato una migrazione da mezz'ora: lanci `/background`, recuperi il terminale e la monitori a parte.
- **NOME COMANDO:** `/tasks`
A COSA SERVE: Vedere e gestire tutto ciò che sta girando in background nella sessione. Il pannello di controllo del lavoro delegato. Disponibile anche come `/bashes`.
UN ESEMPIO CONCRETO: Dopo due `/fork` e un comando lungo, usi `/tasks` per controllare a che punto sono e fermare ciò che non serve.

4. Interazione e portabilità

- **NOME COMANDO:** `/btw`
A COSA SERVE: Fare una domanda al volo ("by the way") mentre Claude esegue un'altra attività, ottenendo la risposta senza interromperlo e senza gonfiare la cronologia.
UN ESEMPIO CONCRETO: Mentre Claude fa girare i test, lanci `/btw` come si chiama l'hook React per il debounce? e ricevi la risposta senza fermare il lavoro.
- **NOME COMANDO:** `/export`
A COSA SERVE: Salvare in un file le risposte di Claude Code come testo semplice. Comodo per conservare ricerche o lo sviluppo di un'idea.
UN ESEMPIO CONCRETO: Dopo aver fatto analizzare tre opzioni di architettura, lanci `/export` `decisione-architettura.txt` per archivarle.
- **NOME COMANDO:** `/remote-control`
A COSA SERVE: Continuare la stessa sessione dallo smartphone, mantenendo accesso a file, MCP server e agenti del computer, con tutto sincronizzato. Richiede l'app di Claude e il computer acceso. Alias: `/rc`.
UN ESEMPIO CONCRETO: Avvii un task lungo, lanci `/remote-control`, esci di casa e dal telefono (sezione Code) ne controlli l'avanzamento.
- **NOME COMANDO:** `/teleport`
A COSA SERVE: Tirare una sessione web di Claude Code dentro il terminale locale, scaricando branch e conversazione. Il gemello opposto di `/remote-control`. Alias: `/tp`.
UN ESEMPIO CONCRETO: Avevi avviato un task dal browser; lanci `/teleport`, lo selezioni e continui a lavorarci dal terminale.

5. Review, verifica e qualità del codice

- **NOME COMANDO:** `/code-review`
A COSA SERVE: Rivedere il diff cercando bug di correttezza più opportunità di pulizia. Con `--fix` applica le correzioni, con `--comment` le posta sulla PR GitHub, con `ultra` lancia una review multi-agente nel cloud.
UN ESEMPIO CONCRETO: Prima del commit lanci `/code-review --fix` e Claude individua un null pointer e lo corregge al volo.

- **NOME COMANDO:** `/simplify`
A COSA SERVE: Rivedere il codice modificato per semplificarlo e renderlo più efficiente, con quattro agenti in parallelo (riuso, semplificazione, efficienza, astrazione). Dalla v2.1.154 non cerca bug: per quelli c'è `/code-review`.
UN ESEMPIO CONCRETO: Hai una funzione lunga e ridondante: lanci `/simplify src/auth/` e Claude la riscrive riutilizzando helper già presenti.
- **NOME COMANDO:** `/security-review`
A COSA SERVE: Analizzare le modifiche del branch in cerca di vulnerabilità: injection, problemi di auth, esposizione di dati. Il terzo del trittico "prima di shippare".
UN ESEMPIO CONCRETO: Prima della PR lanci `/security-review` e Claude segnala una query SQL costruita concatenando input utente, a rischio injection.
- **NOME COMANDO:** `/diff`
A COSA SERVE: Aprire un viewer interattivo per ispezionare le modifiche non committate, anche turno per turno. Per vedere esattamente cosa ha toccato Claude.
UN ESEMPIO CONCRETO: Dopo una sessione fitta lanci `/diff` e, con le frecce, sfogli file per file ciò che è cambiato prima di committare.
- **NOME COMANDO:** `/verify`
A COSA SERVE: Confermare che una modifica funzioni davvero costruendo e lanciando l'app per osservarne il risultato, invece di fidarsi solo dei test.
UN ESEMPIO CONCRETO: Dopo aver sistemato il login lanci `/verify`, Claude avvia l'app, prova il login e conferma che funziona end-to-end.
- **NOME COMANDO:** `/run`
A COSA SERVE: Avviare e pilotare l'app del progetto per vedere una modifica funzionante nell'applicazione reale, non solo nei test. Stretto parente di `/verify`.
UN ESEMPIO CONCRETO: Vuoi controllare una nuova schermata: lanci `/run`, Claude avvia l'app e naviga fino a quella pagina così la vedi dal vivo.
- **NOME COMANDO:** `/autofix-pr`
A COSA SERVE: Avviare una sessione sul web che sorveglia la PR del tuo branch e pusha correzioni automatiche quando la CI fallisce o i reviewer commentano. Richiede la CLI gh.
UN ESEMPIO CONCRETO: Apri una PR, lanci `/autofix-pr only fix lint and type errors`, chiudi il portatile e torni con la PR già verde su lint e tipi.

6. Monitoraggio costi

- **NOME COMANDO:** /usage

A COSA SERVE: Mostrare costo della sessione, limiti del piano e statistiche, con breakdown per skill, subagent, plugin e MCP server. Chiude il tema token: /context ti dice quanto contesto usi, /usage quanto spendi. Alias: /cost, /stats.

UN ESEMPIO CONCRETO: A metà giornata lanci /usage per vedere quanto hai consumato e decidere se scalare su un modello più leggero.

Vuoi lavorare con me? 🙌 <https://antonioguangadagno.it/>